AGENTIC SOFTWARE DEVELOPMENT · TOKEN EFFICIENCY

21 agnostic methods to save tokens

Arranged by the moment in the agent cycle where they take effect: from the plan through context and tool execution to the history that is resent on every turn.

AFFECTS 🟦 Input tokens 🟧 Output tokens 🟩 Price per token 🟪 Iterations / calls

TOP LEVER biggest impact in long sessions

PHASE 0

Plan

How many attempts does the task need?

20 🟪 ITERATIONS **TOP LEVER**

Spec-first / Plan-then-Execute

PROCESS LAYER

Create and approve a plan first, then implement. Every failure loop resends the entire context.

Fewer iterations beat any micro-compression

↓

PHASE 1

Configure the call

Which model, how much thinking, when – and is the call needed at all?

11 🟩 PRICE PER TOKEN

Model Routing / Cascading

ROUTING LAYER

Cheap model for classification and routine edits; escalate to the expensive model only on uncertainty or failure.

40–70 % lower cost

17 🟧 OUTPUT TOKENS

Reasoning Budget Control

MODEL LAYER

Set thinking or effort parameters per task: low for routine work, high for architecture questions.

Reasoning often exceeds the visible answer many times over

18 🟩 PRICE PER TOKEN

Batch APIs

INFRASTRUCTURE LAYER

Submit non-time-critical jobs asynchronously: nightly reviews, docs generation, bulk processing.

around 50 % off with all major providers

16 🟪 CALLS

Semantic Response Caching

INFRASTRUCTURE LAYER

Reuse responses to semantically similar requests via embedding similarity, e.g. with GPTCache or Redis Vector.

Call skipped entirely – rarely applicable to coding agents

↓

PHASE 2

Assemble context

What goes into the prompt – and in what form?

02 🟩 PRICE PER TOKEN

Prefix Alignment & Prompt Caching

INFRASTRUCTURE LAYER

Static content first (system prompt, AGENTS.md, tools), dynamic content last. On Anthropic with explicit breakpoints, default TTL 5 minutes.

up to 90 % cheaper on cache hits

13 🟦 INPUT TOKENS **TOP LEVER**

Dynamic Tool Loading

PROTOCOL LAYER

Load only a tool index and fetch full schemas on demand; disable unused MCP servers per project.

often several 10k tokens per call

06 🟦 INPUT TOKENS

Progressive Disclosure / JIT Retrieval

DATA LAYER

Load docs and architecture knowledge only when a need is detected, e.g. via MCP.

no upfront ballast in the system prompt

05 🟦 INPUT TOKENS

AST-based Repository Mapping

CONTEXT LAYER

Tree-Sitter plus PageRank builds a compact symbol map instead of raw source files.

70–80 % less context for repo analysis

15 🟦 INPUT TOKENS

Token-efficient Data Formats

INPUT LAYER

Feed tabular data as CSV, YAML or TOON instead of pretty-printed JSON.

30–60 % on uniform object arrays

14 🟦 INPUT TOKENS

Prompt Compression

INPUT LAYER

LLMLingua removes low-information tokens based on a small model's perplexity. Good for RAG, risky for code.

works on prose, not on syntax

↓

PHASE 3

Inference & response

What does the model output?

03 🟧 OUTPUT TOKENS

Caveman Pattern

OUTPUT / MODEL LAYER

The system prompt instructs the model to drop filler words, pleasantries and redundant justifications.

65–75 % less prose – smaller overall effect

07 🟧 OUTPUT TOKENS

Strict Response Schemas

PROTOCOL LAYER

Enforce structured responses (JSON, Pydantic, YAML). Put a reasoning field first in the schema, or quality suffers.

only the necessary data format

12 🟧 OUTPUT TOKENS

Diff-based Edits

OUTPUT LAYER

Search/replace blocks or unified diffs instead of re-emitting the whole file.

90 % or more on large files

↓

PHASE 4

Tool execution

What flows back from shell, files and sub-agents?

08 🟦 INPUT TOKENS **TOP LEVER**

Sub-Agent Isolation

ORCHESTRATION LAYER

Delegate code search, log analysis and research to sub-agents with a fresh context. Only the result comes back.

often the single biggest lever

01 🟦 INPUT TOKENS

RTK (Rust Token Killer)

EXECUTION LAYER

Intercepts output from git, pytest, cargo test or docker logs and strips boilerplate before it reaches the context.

60–90 % on shell and tool output

04 🟦 INPUT TOKENS

File Piping & Stream Filtering

OS LAYER

Don't read large logs and files natively; pre-filter them with grep, head and awk.

prevents thousands of tokens of context flood

21 🟦 INPUT TOKENS

Targeted Reading & Read Dedup

CONTEXT LAYER

Read files in sections via line ranges and never read the same file twice.

only relevant lines in context

19 🟪 ITERATIONS

Deterministic Hooks

EXECUTION LAYER

Formatter, linter and import sorting as pre- or post-hooks instead of repair jobs for the agent.

a script replaces a whole turn

↓

PHASE 5

Maintain history

What does the next turn carry along?

09 🟦 INPUT TOKENS **TOP LEVER**

Observation Masking

MEMORY LAYER

Replace old tool outputs after a few turns with short placeholders, e.g. "pytest output removed, 3 failures fixed".

about as effective as summarizing, but cheaper

10 🟦 INPUT TOKENS **TOP LEVER**

Context Compaction

MEMORY LAYER

Summarize the history shortly before the limit; keep decisions and open items in a NOTES.md.

the next turn starts lean

WHY INPUT DOMINATES

Every turn resends the complete history.

In agentic workloads, input therefore often accounts for over 90 % of tokens. Output optimizations help, but less.

Next turn

entire history is resent

Three levers

Fewer tokens

Reduce input and output tokens – the biggest lever is the context.

Cheaper tokens

Caching, routing and batching lower the price without changing the content.

Fewer calls

Planning and hooks avoid loops that resend the history again and again.

Percentages are typical ranges; actual savings depend on workload, model and provider.